

AIMLCZG546 · SESSION 3 · COMPANION READER

Requirements Engineering for ML Systems

When to Use ML, Goals, GR4ML Notation, and Measuring Success

Session 3 · Read alongside the lecture slides · Every concept explained step by step

How to use this companion. **Blue** = core intuition. **Amber** = everyday analogy. **Green** = worked scenario. **Red** = pitfall. **Purple** = the one line to remember. The running example throughout is a *credit risk prediction system* at a bank.

1 When to Use ML

ML is a powerful tool, but it is not the right tool for every problem. Geoff Hulten's framework identifies three situations where ML genuinely outperforms rule-based approaches.

The intuition

A screwdriver is perfect for screws but terrible for nails. ML is perfect for certain types of problems. Using it on the wrong problem wastes months of effort and produces worse results than simpler alternatives.

1.1 Case 1: Intrinsically Hard Problems

Human language understanding is a canonical example. Consider sarcasm: “Oh great, another Monday.” A rule-based system sees the word “great” and classifies this as positive sentiment. A human immediately understands it is negative. Language is contextual, ambiguous, culturally loaded, and constantly evolving. No finite set of hand-crafted rules can capture this. ML models learn the patterns from millions of examples.

1.2 Case 2: Big Problems (Scale)

Recommending the right song from 100 million tracks to 500 million users, based on each user's listening history, mood, and time of day, is mathematically intractable for a human writing rules. ML can learn a personalised embedding space for every user and every song and compute recommendations in milliseconds.

1.3 Case 3: Time-Changing Problems

Fraud patterns evolve. As soon as a rule-based fraud system blocks a particular attack vector, fraudsters adapt. An ML model that retrains weekly on the latest fraud cases continuously adapts to new attack patterns — a fixed rule set does not.

★ Everyday picture

Case 1 is like asking a human to sort mail by handwriting style across a million letters — too subjective for rules. Case 2 is like building a menu for every customer in a billion-person restaurant — too large for manual rules. Case 3 is like a lock that automatically changes its combination when someone figures it out.

✗ Watch out

Using ML on a simple, deterministic problem is a mistake. If a business rule can be written clearly (“if age < 18, block purchase”), write the rule. ML adds cost, complexity, and uncertainty you do not need.

Where this shows up in ML/AI. Every ML project starts with a justification: *why ML and not a rule?* The answer must fall into one of Hulten’s three cases, or the project should be reconsidered.

2 Requirements Engineering for ML Systems

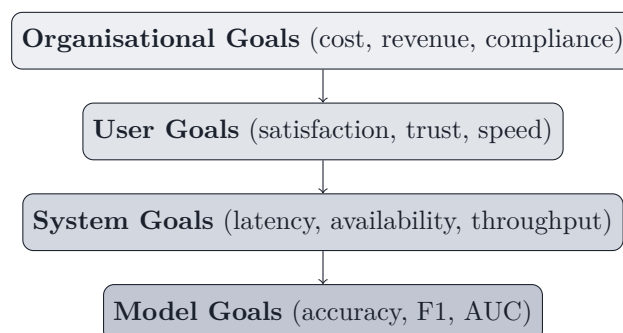
Requirements Engineering (RE) is the process of discovering, documenting, and managing what a system must do and how well it must do it. Traditional RE is straightforward: systems are deterministic. An ML system is probabilistic, data-dependent, and continuously changing, which makes RE significantly harder.

Dimension	Traditional RE	ML RE
Specification	Clear, exhaustive rules	Measurable performance targets on data
Correctness	Binary (passes/fails test)	Probabilistic (correct 93 % of the time)
Data dependency	None	All predictions depend on training data quality
Change	Specification changes when business changes	Specification changes when data distribution changes

RE in ML must address four additional concerns not present in traditional RE: data quality and availability, model behaviour under distributional shift, continuous learning and retraining cycles, and fairness and bias across user populations.

3 Goals in ML Systems

A **goal** is a desired outcome. In an ML system, goals must be aligned across four levels simultaneously.



The intuition

Think of it as a telescope: each lens must be aligned for the final image to be clear. If the model is accurate but the system is slow, users are unhappy. If users are happy but the system is expensive, the business is unhappy. All four lenses must point in the same direction.

Worked example: Goal alignment in the credit risk system

A bank wants to approve loan applications faster while reducing default rates. The goal alignment looks like:

Organisational goal: Reduce loan default rate by 15 % over 12 months; reduce manual review cost by 40 %.

User goal (loan officer): Get a risk score with explanation within 10 seconds of submitting application.

System goal: Process risk scoring API in under 2 seconds at 99.9 % availability.

Model goal: Achieve $AUC \geq 0.85$ on held-out test set; false negative rate $\leq 5\%$ (do not miss true defaults).

Notice: the model goal is *derived* from the organisational and user goals. You cannot define the model goal without knowing the business and user goals.

⚠ Watch out

Optimising only the model goal (AUC) while ignoring system goals (latency) is a classic RE mistake. A model with AUC 0.92 that takes 30 seconds to respond is useless if the loan officer is waiting at the counter with a customer.

4 GR4ML: Business View

GR4ML (Goal and Requirement for ML) is a formal modelling notation that connects high-level business goals to specific ML components. It has three views: Business, Analytics Design, and Data Preparation.

The **Business View** models the organisational context of the ML system.

Element	Meaning	Credit risk example
Actor	Stakeholder who drives decisions	Case Worker (loan officer)
StrategicGoal	High-level business objective	Make good lending decisions
DecisionGoal	Specific decision to fulfill the strategy	Approve or reject this loan application
QuestionGoal	Question that must be answered to decide	What is the credit risk of this applicant?
Insight	Data-driven answer to the question	Credit risk score: 0.73 (high risk)
Indicator	Metric measuring if the goal is achieved	Loan default rate (%)
UpdateFrequency	How often the model runs	Monthly (batch retraining)
LearningPeriod	Historical window used for training	Last 24 months of loan data

The intuition

The Business View is a “why” map. It traces the chain from “why do we need this ML system at all” (StrategicGoal) down to “what specific number does the ML model produce” (Insight). This chain ensures every ML decision can be justified by a business need.

5 GR4ML: Analytics Design View

The **Analytics Design View** specifies what kind of analytical task the ML component performs and which quality attributes guide the algorithm choice.

Element	Meaning	Example
AnalyticsGoal	The type of insight to generate	PredictionGoal: predict default probability
PredictionGoal	Forecast future/unknown outcome	Will this applicant default?
DescriptionGoal	Understand or summarise data	Which customer segments default most?
PrescriptionGoal	Recommend optimal action	Which loan terms minimise expected loss?
Algorithm	Computational method chosen	Gradient Boosted Trees (XGBoost)
SoftGoal (Interpretability)	Loan officers must explain decisions to customers	High weight given to explainability
SoftGoal (Accuracy)	Regulatory requirement: $AUC \geq 0.85$	High weight given to predictive power

Worked example: Analytics design trade-off

The bank's credit risk model must be both *accurate* (minimise defaults) and *interpretable* (loan officers must explain rejections to customers — a regulatory requirement).

XGBoost achieves AUC 0.88 but is a black box. A logistic regression achieves AUC 0.81 but is fully interpretable.

Decision: Use XGBoost with SHAP (SHapley Additive exPlanations) values. SHAP produces an explanation: "This application was rejected primarily because debt-to-income ratio (45 %) exceeds the threshold (35 %), and credit utilisation (82 %) is very high." This satisfies both SoftGoals.

6 GR4ML: Data Preparation View

The **Data Preparation View** maps the raw data sources to the clean, feature-engineered dataset that the model will train on.

Key elements:

- **Entity** — data objects (tables, APIs, files): `LoanApplication`, `CreditBureau`, `TransactionHistory`.
- **DataPreparationTask** — operations: Data Cleaning, Data Reduction, Normalisation, Encoding.
- **Operator** — specific method: remove rows with missing income (cleaning), min-max scale loan amount (normalisation), one-hot encode loan purpose (encoding).
- **Mechanism** — implementation: Python/pandas, Apache Spark, SQL.

Worked example: Data preparation for credit risk

Input: Raw tables — `LoanApplication` (50 columns), `CreditBureau` (30 columns), 3 years of `TransactionHistory`.

Task 1 (Cleaning): Remove 3,200 rows with missing annual income (3.2 % of records). Impute 800 rows with missing credit score using median by age bracket.

Task 2 (Reduction): Drop 22 columns with >40 % missing values. Retain 35 predictive features.

Task 3 (Normalisation): Min-max scale: `loan_amount`, `annual_income`, `total_debt` (to bring all to [0, 1]).

Task 4 (Encoding): One-hot encode: `loan_purpose` (12 categories → 12 binary columns),

employment_type (4 categories).

Output: 97,000 clean rows, 58 features, ready for XGBoost training.

7 Measuring Goals: Measures and Metrics

A **measure** is the method or standard used to quantify something. A **metric** is the actual measured value. A good measure: directly relates to a goal, is quantifiable and objective, and is practical to collect.

★ Everyday picture

A measure for “good health” might be blood pressure, body temperature, and resting heart rate. The metric for your blood pressure today is 120/80 mmHg. The goal (good health) is abstract; the measure (blood pressure) makes it concrete; the metric (120/80) tells you where you stand today.

Worked example: Chatbot goal to measures

Goal: “Improve chatbot usefulness.”

Measure 1: Task success rate — fraction of conversations in which the user’s intent was fulfilled.

Measure 2: User satisfaction score (1–5 stars, collected post-chat).

Measure 3: Conversation completion rate — fraction of users who did not abandon mid-conversation.

Measure 4: Escalation rate — fraction of conversations escalated to a human agent (lower is better).

Notice that “improve chatbot usefulness” is unmeasurable by itself. Only after you define the four measures can you set targets, run experiments, and know if you are improving.

8 Accuracy vs Precision of Measurement

These two words are often confused. They describe independent properties of a measurement system.

- **Accuracy** — how close the measured value is to the true value.
- **Precision** — how consistently the measurement produces the same result when repeated.

The intuition

Think of darts. Accurate = clustered around the bullseye. Precise = clustered tightly together (but may be far from the bullseye). The ideal is both: tight cluster, centred on the bullseye. The worst is neither: scattered all over.

Worked example: Thermometer accuracy and precision

True body temperature: 98.6°F. We take five measurements with two thermometers:

Thermometer A (precise, not accurate): [98.2, 98.1, 98.3, 98.2, 98.1]

Mean = 98.18°F. Bias = −0.42°F. Std dev = 0.08°F.

Verdict: very consistent (low std dev) but systematically off (high bias). Calibration can fix this.

Thermometer B (accurate, not precise): [98.4, 99.1, 97.9, 98.8, 99.2]

Mean = 98.68°F. Bias = +0.08°F. Std dev = 0.53°F.

Verdict: centred on the right value (low bias) but noisy (high std dev). Hard to rely on for a single reading.

In ML: A model with high precision/low variance makes consistent predictions; high accuracy/low bias means those predictions are correct. Both matter. Bias–variance trade-off is the ML counterpart of accuracy–precision.

✓ **Key takeaway**

Precision without accuracy is measuring the wrong thing reliably. Accuracy without precision means you cannot trust any single measurement. You need both.